

1. Heuristic design and admissibility

a. Steps heuristic

- Formula: $h_{\text{steps}}(n)$ is equal to the number of positions where state(n) and the goal word are different, referred to as the Hamming distance.
- Lower bound: Every action can only fix one position, meaning that if three positions were wrong, there are at least three more actions that need to occur
- It can't overestimate because every action costs 1, and it counts only the minimum number of actions that still need to happen. This means that it's never higher than the real cost, and once it reaches the goal, it's exactly zero.

b. Scrabble heuristic

- Formula: $h_{\text{scrabble}}(n) = \text{sum of scrabbleValue(goal[i]) for every position } i \text{ that state}(n) \text{ and the goal are different}$
- Lower bound: For the wrong letter at position i , it must eventually be replaced with the goal's letter at that position, which is the last time position i will change. That action costs exactly the Scrabble value of the goal letter.
- It can't overestimate because we only add the exact needed costs: one per mismatched position. If the search needs extra detours, it can only add to the cost, never reduce it. In the end, nothing is mismatched, making the sum 0.

c. Frequency heuristic

- Formula: $n = \text{the Hamming distance from state}(n_{\text{node}}) \text{ to the goal}$. $H_{\text{freq}}(n) = 0$ under the condition that $n=0$, otherwise $n + \text{rarity(goal)}$
- Lower bound: like steps, at least n actions would remain. Additionally, the final action of any solution must produce the goal word itself, and that action will always cost $1 + \text{rarity(goal)}$. Any other remaining action would cost at least 1 because rarity is never negative.
- It can't overestimate because when we add up the $n-1$ actions, which cost at least 1 each, and the guaranteed last action, which costs exactly $1 + \text{rarity(goal)}$, we get $n + \text{rarity(goal)}$ as a safe lower bound. It's a better bound than the plain Hamming distance because it uses the one cost we know 100 percent: the cost of the final step.

2.

Word pair	Cost measure	UCS COST	UCS nodes expanded	A*cost	A* nodes expanded	Reduction
WHY -> ASK	Steps	5.00	773	5.00	16	97.93%
WHY -> ASK	Scrabble	9.00	765	9.00	13	98.30%
WHY -> ASK	Frequency	38.86	808	38.86	634	21.53%
BEAR -> DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR -> DUCK	Scrabble	11.00	2981	11.00	6	99.80%
BEAR -> DUCK	Frequency	38.24	1084	38.24	302	72.14%

- b. Yes, UCS and A* reported identical solution costs, as seen through example such as UCS and A* cost for Why -> ASK Steps both being 5.00 or both cost for WHY -> ASK Scrabble being 9.00.
- c. For WHY -> ASK, the largest reduction was Scrabble with a 98.30% reduction. Scrabble was also the highest reduction for BEAR -> DUCK, with a 99.80% reduction
- d. For both WHY -> ASK and BEAR -> DUCK, Frequency was the least informative heuristic. For WHY -> ASK it was a 21.53% reduction and for BEAR -> DUCK it was a 72.14% reduction.
- e. Both steps and scrabble look at how both how many letters are still wrong and how expensive that fix will be, giving A* a reliable signal towards the goal, allowing it to skip most of the nodes that UCS would still have to check. Frequency is weaker because its estimate $n + \text{rarity}$ only knows the rarity of the final goal word, not the rarity of the in-between words the search must

pass through, which can vary a lot. Cost heavily depends on those in between words, so the frequency heuristic underestimates by a unpredictable huge margin, meaning that A* still checks just about as many nodes as NCS

3.

- a. Yes, every action can be undone by another legal action, and each action will cost one step. This means reversing an optimal path forward gives an equally short optimal path backward.
- b. No, the cost depends not on the letter being removed, but on the letter being added. This means that going one way could cost more than going back. For example, “cat” and “hat” are different by one letter. Getting from cat to hat involves adding ‘h’, which costs 4, but getting from hat to cat adds ‘c’, which costs 3. Even though it’s the same edge, the different directions have different costs.
- c. No, the cost of an action is $1 + \text{rarity}$, so it fully depends on which word you're moving into. For example, going from cat to hat costs $1 + \text{rarity}(\text{'hat'}) = 7.45$, but going from hat to cat costs $1 + \text{rarity}(\text{'cat'}) = 7.89$. As shown in the example, different destinations have different costs.
- d. Even though an action is always reversible, it does not mean it’s the same cost both ways. Steps doesn’t worry about letters so that it can stay symmetric, but Scrabble and frequency care about which word or letter is involved, and that factor changes depending on which way you're going.

4.

a. Model: Gemini 3.8 flash, prompt: “in DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.” Proposed heuristic “Let n be the current word state and g be the

goal word state. Let $\mathcal{D}$ be the word dictionary, where each word $w \in \mathcal{D}$ has a non-negative rarity value $\text{rarity}(w) \geq 0$."

b. Formula: $h_1(n) = h_0(n) \cdot (1 + r_{\min})$

c. The heuristic is admissible. This is because of two strict minimum bounds, firstly every transition that's valid changes at most one letter, meaning reaching g from n requires at least $m \geq h_0(n)$ transition. Secondly, every transition to word w costs $1 + \text{rarity}(w)$ and because $r_{\min} = \min_{w \in \mathcal{D}} \text{rarity}(w)$, there is no valid state in the dictionary that can cost less than $1 + r_{\min}$. The total cost for any path of length m is $\sum_{k=1}^m c(w_k) \geq m(1 + r_{\min}) \geq h_0(n)(1 + r_{\min}) = h_1(n)$. Every single step is bounded by $1 + r_{\min}$ and the number of steps is bounded by $h_0(n)$, meaning that the product will never overestimate the true cost $h^*(n)$, even if a edge case were to occur such as if $r_{\min} = 0$ or detours were needed.

d. The heuristic is valid and doesn't need repairs. Even in a edge case where $r_{\min} = 0$, $h_1(n)$ will safely collapse to standard Hamming distance $h_0(n)$, which will preserve admissibility across any dictionary configuration

e.

Problem	Heuristic	Solution Cost	Nodes Expanding
WHY -> ASK	Baseline(h_0)	38.855	755
	Proposed(h_1)	38.855	754
BEAR -> DUCK	Baseline(h_0)	38.240	720
	Proposed(h_1)	38.240	714

When comparing, both h_0 and h_1 showed the same solution costs: for WHY -> ASK, both had a solution cost of 38.855, and for BEAR -> DUCK, both had a solution cost of 38.240. However, the proposed heuristics always expanded to fewer states than the baseline, expanding one fewer node for WHY -> ASK and 6 fewer nodes for BEAR -> DUCK. This evidence supports using the proposed heuristic.

AI disclosure

Grammarly was used for all written questions, and Claude was used for an issue where I didn't have NumPy installed, so my code wouldn't run.